

**Московский авиационный институт  
(национальный исследовательский университет)**

**Факультет информационных технологий и прикладной математики**

**Кафедра вычислительной математики и программирования**

**Лабораторная работа №5 по курсу «Дискретный анализ»**

Студент: Р. А. Савицкас  
Преподаватель: А. А. Кухтичев  
Группа: М8О-206БВ-24  
Дата:  
Оценка:  
Подпись:

**Москва, 2026**

# Лабораторная работа №5

## Задание

Необходимо реализовать поиск образцов в заранее известном тексте с использованием суффиксного массива.

На вход программе подаётся текст, расположенный на первой строке. Далее, до конца файла, следуют строки с образцами, которые необходимо найти в этом тексте.

Для каждого найденного образца требуется вывести строку, начинающуюся с последовательного номера данного образца и двоеточия. После двоеточия через запятую должны быть перечислены номера позиций, в которых данный образец встречается в тексте. Позиции необходимо выводить в порядке возрастания.

Ограничения задачи:

- ограничение времени: 1 секунда;
- ограничение памяти: 1280 Мб;
- ввод: стандартный ввод или `input.txt`;
- вывод: стандартный вывод или `output.txt`.

Пример входных данных:

```
abcdabc abcd bcd bc
```

Пример выходных данных:

```
1: 1 2: 2 3: 2, 6
```

В работе используется подход на основе суффиксного массива. Для ускорения поиска дополнительно применяются массив LCP и структура RMQ, позволяющая быстро находить длину наибольшего общего префикса двух суффиксов. Это позволяет уменьшить количество повторных сравнений символов при бинарном поиске образца в суффиксном массиве.

Результатом лабораторной работы является программа, которая:

- строит суффиксный массив для заданного текста;
- строит массив LCP;
- строит структуру RMQ для быстрых запросов минимума на отрезке массива LCP;
- для каждого образца находит диапазон суффиксов, начинающихся с данного образца;
- выводит все позиции вхождений образца в порядке возрастания.

# 1 Исходный код

Ключевые особенности реализации:

- **Суффиксный массив:** для исходного текста строится суффиксный массив методом удвоения. Для корректного построения к строке добавляется терминальный символ `'\0'`, который лексикографически меньше остальных символов.
- **Массив LCP:** после построения суффиксного массива строится массив наибольших общих префиксов соседних суффиксов. Это позволяет избежать повторного сравнения одних и тех же символов при поиске образцов.
- **RMQ:** над массивом LCP строится Sparse Table для быстрых запросов минимума на отрезке. Это позволяет за  $O(1)$  получать LCP двух произвольных суффиксов, зная их позиции в суффиксном массиве.
- **Оптимизация поиска:** поиск границы диапазона суффиксов выполняется с использованием LCP-ускоренного бинарного поиска. За счёт хранения уже известного LCP с левой и правой границами уменьшается количество посимвольных сравнений.
- **Оптимизация вывода:** вывод накапливается в строковом буфере, после чего записывается крупными блоками. Это уменьшает накладные расходы на большое количество операций вывода.

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 string s;
5 vector<int> sa, rnk, lcp, lg;
6 vector<vector<int>> st;
7
8 void trim_cr(string& x) {
9     if (!x.empty() && x.back() == '\r') x.pop_back();
10 }
11
12 vector<int> build_sa(string t) {
13     t += '\0';
14
15     int n = t.size();
16     vector<int> p(n), c(n), pn(n), cn(n), cnt(max(256, n), 0);
17
18     for (unsigned char ch : t) cnt[ch]++;
19     for (int i = 1; i < 256; ++i) cnt[i] += cnt[i - 1];
20     for (int i = 0; i < n; ++i) p[--cnt[(unsigned char)t[i]]] = i;
21
22     int cls = 1;
23     c[p[0]] = 0;
24
25     for (int i = 1; i < n; ++i) {
26         if (t[p[i]] != t[p[i - 1]]) ++cls;
27         c[p[i]] = cls - 1;
28     }
29
30     for (int len = 1; len < n && cls < n; len <= 1) {
```

```

31     for (int i = 0; i < n; ++i) {
32         pn[i] = p[i] - len;
33         if (pn[i] < 0) pn[i] += n;
34     }
35
36     fill(cnt.begin(), cnt.begin() + cls, 0);
37
38     for (int x : pn) cnt[c[x]]++;
39     for (int i = 1; i < cls; ++i) cnt[i] += cnt[i - 1];
40
41     for (int i = n - 1; i >= 0; --i) {
42         int x = pn[i];
43         p[--cnt[c[x]]] = x;
44     }
45
46     cn[p[0]] = 0;
47     int ncls = 1;
48
49     for (int i = 1; i < n; ++i) {
50         pair<int, int> cur = {c[p[i]], c[(p[i] + len) % n]};
51         pair<int, int> prv = {c[p[i - 1]], c[(p[i - 1] + len) % n]};
52         // ↗ ]};
53         if (cur != prv) ++ncls;
54         cn[p[i]] = ncls - 1;
55     }
56
57     c.swap(cn);
58     cls = ncls;
59 }
60
61 p.erase(p.begin());
62 return p;
63 }
64
65 void build_lcp() {
66     int n = s.size();
67
68     rnk.assign(n, 0);
69     lcp.assign(n, 0);
70
71     for (int i = 0; i < n; ++i) {
72         rnk[sa[i]] = i;
73     }
74
75     int k = 0;
76
77     for (int i = 0; i < n; ++i) {
78         int pos = rnk[i];
79
80         if (pos == 0) continue;
81
82         int j = sa[pos - 1];

```

```

83
84     while (i + k < n && j + k < n && s[i + k] == s[j + k]) {
85         ++k;
86     }
87
88     lcp[pos] = k;
89
90     if (k) --k;
91 }
92 }
93
94 void build_rmq() {
95     int n = lcp.size();
96
97     lg.assign(n + 1, 0);
98
99     for (int i = 2; i <= n; ++i) {
100         lg[i] = lg[i / 2] + 1;
101     }
102
103     st.assign(lg[n] + 1, vector<int>(n));
104     st[0] = lcp;
105
106     for (int k = 1; k < (int)st.size(); ++k) {
107         int len = 1 << k;
108         int half = len >> 1;
109
110         for (int i = 0; i + len <= n; ++i) {
111             st[k][i] = min(st[k - 1][i], st[k - 1][i + half]);
112         }
113     }
114 }
115
116 int get_lcp(int i, int j) {
117     if (i == j) return (int)s.size() - sa[i];
118
119     if (i > j) swap(i, j);
120
121     ++i;
122
123     int len = j - i + 1;
124     int k = lg[len];
125
126     return min(st[k][i], st[k][j - (1 << k) + 1]);
127 }
128
129 pair<int, int> cmp_suffix(int pos, const string& p, int known) {
130     int n = s.size();
131     int m = p.size();
132     int i = known;
133
134     while (i < m && pos + i < n && s[pos + i] == p[i]) {
135         ++i;

```

```

136     }
137
138     if (i == m && pos + i == n) return {0, i};
139     if (i == m) return {1, i};
140     if (pos + i == n) return {-1, i};
141
142     return {(unsigned char)s[pos + i] < (unsigned char)p[i] ? -1 : 1, i
143             ↪ };
144 }
145
146 int lower_bound_lcp(const string& p) {
147     int n = sa.size();
148
149     auto lc = cmp_suffix(sa[0], p, 0);
150     if (lc.first >= 0) return 0;
151
152     auto rc = cmp_suffix(sa[n - 1], p, 0);
153     if (rc.first < 0) return n;
154
155     int l = 0;
156     int r = n - 1;
157     int lcp_l = lc.second;
158     int lcp_r = rc.second;
159
160     while (r - l > 1) {
161         int m = (l + r) / 2;
162
163         if (lcp_l >= lcp_r) {
164             int x = get_lcp(l, m);
165
166             if (x > lcp_l) {
167                 l = m;
168             } else if (x < lcp_l) {
169                 r = m;
170                 lcp_r = x;
171             } else {
172                 auto c = cmp_suffix(sa[m], p, lcp_l);
173
174                 if (c.first < 0) {
175                     l = m;
176                     lcp_l = c.second;
177                 } else {
178                     r = m;
179                     lcp_r = c.second;
180                 }
181             }
182         } else {
183             int x = get_lcp(m, r);
184
185             if (x > lcp_r) {
186                 r = m;
187             } else if (x < lcp_r) {
188                 l = m;

```

```

188         lcp_l = x;
189     } else {
190         auto c = cmp_suffix(sa[m], p, lcp_r);
191
192         if (c.first < 0) {
193             l = m;
194             lcp_l = c.second;
195         } else {
196             r = m;
197             lcp_r = c.second;
198         }
199     }
200 }
201 }
202
203 return r;
204 }
205
206 bool next_key(const string& p, string& q) {
207     q = p;
208
209     for (int i = (int)q.size() - 1; i >= 0; --i) {
210         unsigned char c = q[i];
211
212         if (c < 255) {
213             q[i] = char(c + 1);
214             q.resize(i + 1);
215             return true;
216         }
217     }
218
219     return false;
220 }
221
222 void add_int(string& out, int x) {
223     char buf[16];
224     int len = 0;
225
226     if (x == 0) {
227         buf[len++] = '0';
228     } else {
229         char rev[16];
230         int cnt = 0;
231
232         while (x) {
233             rev[cnt++] = char('0' + x % 10);
234             x /= 10;
235         }
236
237         while (cnt) {
238             buf[len++] = rev[--cnt];
239         }
240     }

```

```

241
242     out.append(buf, buf + len);
243 }
244
245 int main() {
246     ios::sync_with_stdio(false);
247     cin.tie(nullptr);
248
249     if (!getline(cin, s)) return 0;
250     trim_cr(s);
251
252     if (s.empty()) return 0;
253
254     sa = build_sa(s);
255     build_lcp();
256     build_rmq();
257
258     string p;
259     vector<int> ans;
260     string out;
261
262     out.reserve(1 << 20);
263
264     for (int num = 1; getline(cin, p); ++num) {
265         trim_cr(p);
266
267         if (p.empty() || p.size() > s.size()) continue;
268
269         int first = lower_bound_lcp(p);
270
271         string q;
272         int last = next_key(p, q) ? lower_bound_lcp(q) : (int)sa.size()
                ↪ ;
273
274         if (first >= last) continue;
275
276         ans.clear();
277         ans.reserve(last - first);
278
279         for (int i = first; i < last; ++i) {
280             ans.push_back(sa[i] + 1);
281         }
282
283         sort(ans.begin(), ans.end());
284
285         add_int(out, num);
286         out += ": ";
287
288         for (int i = 0; i < (int)ans.size(); ++i) {
289             if (i) out += ", ";
290             add_int(out, ans[i]);
291         }
292

```



```
293         out += '\n';
294
295         if (out.size() > (1 << 20)) {
296             cout << out;
297             out.clear();
298         }
299     }
300
301     cout << out;
302
303     return 0;
304 }
```

## 2 Исследование производительности и качества

### 2.1 Дневник выполнения работы

В соответствии с заданием, ниже представлен протокол выполнения работы по дням.

| Дата       | Описание проделанной работы                                                                            | Результат                                                                                                                       |
|------------|--------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------|
| 15.05.2026 | Изучение условия задачи и выбор структуры данных для поиска образцов в заранее известном тексте.       | В качестве основной структуры данных выбран суффиксный массив.                                                                  |
| 16.05.2026 | Реализация построения суффиксного массива методом удвоения.                                            | Получена первая рабочая версия программы, выполняющая поиск образцов с помощью бинарного поиска по суффиксному массиву.         |
| 17.05.2026 | Добавление массива LCP и структуры RMQ для ускорения поиска. Реализован LCP-ускоренный бинарный поиск. | Поиск образца оптимизирован до оценки $O(m + \log n)$ , где $m$ — длина образца, $n$ — длина текста.                            |
| 18.05.2026 | Проведение экспериментального исследования времени работы поиска при увеличении длины текста.          | Получены результаты, подтверждающие медленный, близкий к логарифмическому, рост времени поиска при фиксированной длине образца. |

### 2.2 Описание проведённого бенчмарка

Для исследования производительности была написана отдельная программа-бенчмарк. Она строит суффиксный массив, массив LCP и структуру RMQ, после чего выполняет серию поисковых запросов.

В данном эксперименте длина образца была зафиксирована:

$$m = 64.$$

Длина текста  $n$  увеличивалась в два раза на каждом шаге:

$$10000, 20000, 40000, \dots, 1280000.$$

Для каждого значения  $n$  выполнялось 10000 запросов. Время вывода и сортировки позиций не учитывалось, так как исследовалась именно сложность поиска диапазона суффиксов, начинающихся с заданного образца.

Ожидаемая сложность поиска одного образца:

$$O(m + \log n).$$

Так как  $m$  фиксировано, при увеличении  $n$  время поиска должно расти медленно, примерно как  $\log n$ .

## 2.3 Результаты замеров

Ниже приведены результаты запуска бенчмарка командой:

make by-n

| $n$     | build, ms | search, ms | $\mu s / query$ | $\log_2 n$ | $m + \log_2 n$ | $\frac{\mu s}{m + \log n}$ |
|---------|-----------|------------|-----------------|------------|----------------|----------------------------|
| 10000   | 0.883     | 1.179      | 0.117873        | 13.288     | 77.288         | 0.00152512                 |
| 20000   | 1.804     | 1.341      | 0.134118        | 14.288     | 78.288         | 0.00171314                 |
| 40000   | 3.792     | 1.443      | 0.144306        | 15.288     | 79.288         | 0.00182003                 |
| 80000   | 8.011     | 1.440      | 0.143966        | 16.288     | 80.288         | 0.00179313                 |
| 160000  | 17.462    | 1.496      | 0.149647        | 17.288     | 81.288         | 0.00184096                 |
| 320000  | 37.063    | 1.510      | 0.150978        | 18.288     | 82.288         | 0.00183476                 |
| 640000  | 80.995    | 1.544      | 0.154412        | 19.288     | 83.288         | 0.00185396                 |
| 1280000 | 171.639   | 1.564      | 0.156424        | 20.288     | 84.288         | 0.00185583                 |

## 2.4 Анализ результатов

По результатам эксперимента видно, что время построения структур данных растёт значительно быстрее времени поиска. Это ожидаемо, так как суффиксный массив строится методом удвоения, имеющим сложность:

$$O(n \log n).$$

Дополнительно строятся массив LCP и структура RMQ. Эти этапы выполняются один раз для всего текста, после чего программа может обрабатывать множество поисковых запросов.

Наиболее важной частью эксперимента является время поиска. При увеличении длины текста с 10000 до 1280000, то есть в 128 раз, среднее время одного запроса увеличилось только с

$$0.117873 \mu s$$

до

$$0.156424 \mu s.$$

Таким образом, время одного поиска увеличилось примерно в

$$\frac{0.156424}{0.117873} \approx 1.33$$

раза. Если бы поиск зависел от длины текста линейно, то при таком увеличении размера входных данных время должно было бы вырасти примерно в 128 раз. На практике наблюдается намного более медленный рост, что соответствует логарифмической зависимости от  $n$ .

Также была рассчитана величина:

$$\frac{T}{m + \log_2 n}.$$

Начиная с  $n = 40000$ , это значение остаётся практически постоянным:

$$0.00182003, 0.00179313, 0.00184096, 0.00183476, 0.00185396, 0.00185583.$$

Это показывает, что экспериментальное время поиска хорошо согласуется с моделью:

$$T(n, m) = C \cdot (m + \log n).$$

## 2.5 Выводы о производительности

В ходе исследования были получены следующие результаты:

1. Построение суффиксного массива, массива LCP и RMQ требует заметного времени, однако выполняется только один раз для заранее известного текста.
2. После построения вспомогательных структур поиск каждого образца выполняется быстро.
3. При фиксированной длине образца  $m = 64$  и увеличении длины текста  $n$  в 128 раз среднее время одного запроса увеличилось только примерно в 1.33 раза.
4. Отношение  $\frac{T}{m + \log_2 n}$  остаётся почти постоянным, что экспериментально подтверждает оценку времени поиска  $O(m + \log n)$ .
5. Использование LCP и RMQ позволяет избежать повторного сравнения одних и тех же префиксов при бинарном поиске по суффиксному массиву.

## 2.6 Выводы о найденных недочётах

В процессе разработки были выявлены следующие особенности и недочёты первоначального подхода:

1. **Обычный бинарный поиск по суффиксному массиву.** В первой версии сравнение образца с суффиксом могло каждый раз начинаться с первого символа. В худшем случае это приводило к оценке  $O(m \log n)$  на один запрос.
2. **Недостаточное использование информации о соседних суффиксах.** Суффиксный массив сам по себе хранит только порядок суффиксов, но не хранит информацию об их общих префиксах. Для устранения этого недостатка был добавлен массив LCP.
3. **Необходимость быстро получать LCP произвольных суффиксов.** Для LCP-ускоренного бинарного поиска требуется быстро находить наибольший общий префикс двух суффиксов по их позициям в суффиксном массиве. Для этого над массивом LCP была построена структура RMQ.
4. **Большое количество операций вывода.** Так как для одного образца может существовать много вхождений, вывод позиций может быть объёмным. Для уменьшения накладных расходов вывод накапливается в строковом буфере.

### 3 Выводы

В ходе работы я пришёл к следующим выводам:

- **Эффективность суффиксного массива:** Суффиксный массив оказался подходящей структурой данных для задачи поиска множества образцов в заранее известном тексте. После однократного построения структуры можно быстро обрабатывать большое количество запросов.
- **Роль массива LCP:** Использование массива LCP позволило избежать повторного сравнения одних и тех же префиксов при бинарном поиске по суффиксному массиву. В обычной реализации поиск мог занимать  $O(m \log n)$ , так как на каждом шаге бинарного поиска образец мог сравниваться с суффиксом с самого начала. Добавление LCP-оптимизации улучшило оценку поиска до  $O(m + \log n)$ .
- **Использование RMQ:** Для быстрого получения LCP двух произвольных суффиксов была построена структура RMQ на основе Sparse Table. Это позволило отвечать на запросы минимума на отрезке массива LCP за  $O(1)$ , что необходимо для LCP-ускоренного бинарного поиска.
- **Экспериментальное подтверждение сложности:** В ходе бенчмарка длина текста увеличивалась с 10000 до 1280000, то есть в 128 раз, при фиксированной длине образца  $m = 64$ . При этом среднее время одного поиска выросло только с 0.117873 до 0.156424 микросекунды. Это показывает, что поиск не зависит от длины текста линейно и согласуется с оценкой  $O(m + \log n)$ .
- **Стоимость предварительной обработки:** Построение суффиксного массива, массива LCP и RMQ требует дополнительного времени и памяти. Однако эта стоимость оправдана, так как все структуры строятся один раз для заранее известного текста, после чего используются для обработки всех последующих образцов.
- **Практические особенности реализации:** Помимо асимптотической сложности, важную роль играют детали реализации: сортировка найденных позиций для вывода в возрастающем порядке, буферизация вывода и аккуратная работа с индексами. Эти детали не меняют основной идеи алгоритма, но влияют на прохождение задачи в условиях ограничения времени.

Данный опыт закрепил понимание того, что при решении задач на строки важно учитывать не только саму структуру данных, но и вспомогательные алгоритмы вокруг неё. Суффиксный массив сам по себе позволяет выполнять поиск образцов, однако использование LCP и RMQ делает этот поиск теоретически и практически более эффективным. Также работа показала, что экспериментальные замеры полезны для проверки того, насколько реальное поведение программы соответствует заявленной асимптотической оценке.

## Список литературы

- [1] Дэн Гасфилд. *Строки, деревья и последовательности в алгоритмах: информатика и вычислительная биология*. — Cambridge University Press, 1997.
- [2] Томас Х. Кормен, Чарльз И. Лейзерсон, Рональд Л. Ривест, Клиффорд Штайн. *Алгоритмы: построение и анализ*. — 3-е изд. — М. : Вильямс, 2013. — 1328 с.
- [3] Manber U., Myers G. *Suffix arrays: A new method for on-line string searches*. SIAM Journal on Computing, 22(5), 1993. — P. 935–948.
- [4] Kasai T., Lee G., Arimura H., Arikawa S., Park K. *Linear-Time Longest-Common-Prefix Computation in Suffix Arrays and Its Applications*. Proceedings of the 12th Annual Symposium on Combinatorial Pattern Matching, 2001. — P. 181–192.
- [5] Дональд Э. Кнут. *Искусство программирования, том 3. Сортировка и поиск*. — 2-е изд. — М. : Вильямс, 2000. — 832 с.
- [6] *Suffix Array — Algorithms for Competitive Programming* [Электронный ресурс]. URL: <https://cp-algorithms.com/string/suffix-array.html> (дата обращения: 18.05.2026).
- [7] *Sparse Table — Algorithms for Competitive Programming* [Электронный ресурс]. URL: [https://cp-algorithms.com/data\\_structures/sparse-table.html](https://cp-algorithms.com/data_structures/sparse-table.html) (дата обращения: 18.05.2026).
- [8] *C++ reference* [Электронный ресурс]. URL: <https://en.cppreference.com> (дата обращения: 18.05.2026).